

Git Training — Speaker's Handbook

Companion to *Git-Training-By-Sarvesh-Gupta.pptx* · 60 minutes · mixed audience (devs + non-devs) · virtual

How to use this handbook

Read **Part 1** (concept brush-up) once, properly, a day before. That's the part that makes you sound like you know Git rather than like you're reading slides. Everything after that is lookup material: Part 2 is a script per slide, Parts 3–4 are the demos, Part 5 is spare content if you finish early, Part 6 is the Q&A bank.

You do not need to memorise anything. Keep this open on a second monitor.

PART 1 — What you actually need to understand

You said you only use push/pull daily. That's fine — 80% of professional developers only use six commands. But there are five ideas that, once you hold them properly, let you answer almost any question in the room. Spend 30 minutes here.

1.1 A commit is a snapshot, not a diff

Most people assume Git stores "the changes you made." It doesn't. **Every commit stores a complete snapshot of every file in the project at that moment.** Git is just very good at compressing the unchanged parts, so a thousand commits of a small project takes almost no space.

Why this matters: it explains why checking out an old commit is instant (Git just unpacks that snapshot), why Git can't easily lose your history, and why "Git is a database of your project's states" is a better mental model than "Git is a log of edits."

Each commit contains:

- The snapshot of all files
- The author name and email (from your `git config`)
- A timestamp
- The commit message
- A pointer to its **parent** commit (the one before it)

That parent pointer is the whole trick. A chain of commits, each pointing backwards to its parent, *is* your history. A merge commit is simply a commit with two parents.

Say it in the room like this: "Think of each commit as a photograph of the entire project, plus a note saying which photograph came before it. History is just that chain of photos."

1.2 The SHA — that ugly string of letters and numbers

Every commit has an ID like `a1b2c3d4e5f6...` — 40 hexadecimal characters. That's a **SHA-1 hash**: a fingerprint calculated from the commit's contents (the files, the message, the author, the parent).

Three consequences worth knowing:

- Change anything about a commit — even one character of the message — and the SHA changes completely. That's why "rewriting history" is a real thing: you don't edit a commit, you create a replacement with a different ID.
- Because the SHA includes the parent's SHA, tampering with an old commit invalidates every commit after it. Git history is effectively tamper-evident.
- You almost never type the full 40 characters. The first 7 are unique enough: `git show a1b2c3d`.

1.3 HEAD, branches, and why branches are "cheap"

This is the concept that unlocks everything on slides 11 and 12.

- A **branch** is not a copy of your code. It is a 41-byte text file containing one SHA. That's literally all it is — a sticky note that says "this branch currently points at commit a1b2c3d."
- **HEAD** is a pointer to *which branch you're currently on*. When you run `git switch main`, you're moving HEAD to point at `main`.
- When you commit, two things happen: a new commit is created with the current commit as its parent, and the branch sticky-note is moved forward to the new commit.

So "creating a branch" means "writing a 41-byte file." That's why the slide says branches are free. Creating a branch in Git is genuinely faster than creating a folder.

The analogy that lands with non-devs: a branch is a bookmark in a book. Making a new bookmark doesn't copy the book. `git switch` is just moving to a different bookmark.

If someone asks "what's a detached HEAD?" — it means HEAD is pointing directly at a commit rather than at a branch. You get it when you `git checkout <sha>`. Commits you make there belong to no branch, so they're easy to lose. The fix is `git switch -c new-branch-name` to attach a branch to where you are, or `git switch main` to go back.

1.4 "Distributed" — what it actually buys you

Older systems (SVN, TFS) kept the history on a central server. You had the latest files; the server had the history. No server, no history, no commits, no diffs.

Git gives every clone the **entire history**. When you `git clone`, you download every commit ever made. Consequences:

- `git log`, `git diff`, `git commit`, `git branch` all work on a plane with no wifi.
- They're also instant, because nothing crosses a network.
- If GitHub vanished tonight, any developer's laptop could restore the project completely.
- The cost: the first clone of a huge, old repository can be slow, and you have to consciously *synchronise* (push/pull), which is where merge

conflicts come from.

Say it like this: "GitHub isn't the source of truth because it's special. It's the source of truth because we all agreed it is. Technically, every laptop in this call has an equally valid full copy."

1.5 The remote, origin, and tracking branches

When you clone a repo, Git saves the URL you cloned from under the nickname `origin`. There's nothing magic about the word — it's just the default nickname for "the place I got this from." You can have several remotes with different names.

Git also keeps a set of read-only **remote-tracking branches** named `origin/main`, `origin/develop`, and so on. These are Git's memory of *where those branches were on the server the last time you talked to it*.

That's the key to slide 15:

- `git fetch` updates `origin/main` — Git's memory of the server. Your own `main` is untouched. Nothing in your working files changes.
- `git merge origin/main` then brings that into your branch.
- `git pull` is just those two commands back-to-back.

So when someone says "I pulled and it broke my files" — pull is the one that touches your working tree. Fetch never does. That distinction alone is worth a minute of the session.

`git push -u origin feature/x` — the `-u` sets up the link between your local `feature/x` and the remote one, so that afterwards plain `git push` and `git pull` know where to go. That's why you only need `-u` the first time you push a new branch.

1.6 Merge vs rebase, in one paragraph you can defend

- Merge** takes two branches and creates a new commit with two parents. It's honest — the history shows exactly what happened, including the fact that work happened in parallel. It never rewrites anything.
- Rebase** takes your commits, sets them aside, moves your branch to the tip of the target branch, and replays your commits on top one by one. The result is a clean straight line, but the replayed commits are *new commits with new SHAs*. The originals are discarded.

The rule that keeps teams safe: **rebase your own local branch freely; never rebase a branch someone else has pulled**. If you rewrite commits others already have, their history and yours no longer agree, and the next person to push gets a mess.

If asked which your team should use, the safe answer is: "Merge into `main` always. Rebase locally to tidy up your own branch before opening a PR, if you're comfortable. And most teams let the platform do it — GitHub's 'Squash and merge' button gives you a clean line without anyone touching rebase."

1.7 The four areas, said slowly (slide 6)

This is the single most important slide in your deck. Get this right and half the questions never get asked.

Area	What it is	How things get in
Working directory	The actual files on your disk, as your editor sees them	You edit and save
Staging area (index)	A shopping basket for the next commit	<code>git add</code>
Local repository	The <code>.git</code> folder — every commit, on your machine	<code>git commit</code>
Remote repository	GitHub/GitLab — the shared copy	<code>git push</code>

Why the staging area exists at all — this is the question devs ask and most trainers dodge. Answer: because you often change more than one thing before you're ready to commit. Staging lets you commit *only the login fix*, leave the half-done refactor uncommitted, and keep the two changes as separate commits. It's the difference between "commit everything I've touched" and "commit this one idea."

Non-dev analogy for the whole slide: working directory = your desk, covered in papers. Staging = the envelope you're filling. Commit = sealing and dating the envelope and filing it in your cabinet. Push = couriering a copy to head office.

PART 2 — Slide-by-slide speaking guide

Each slide gets: the one idea, roughly what to say, and what usually gets asked. Aim for the times on your agenda slide; they're realistic.

Slides 1-2 • Welcome & agenda (2 min)

Don't over-invest here. Introduce yourself in one line. Then set two expectations that matter for a mixed room:

"Some of you write code every day, some of you have never opened a terminal. I've built this so nobody's lost — the first half is concepts that apply to anyone who works with files, and I'll flag clearly when something is developer-only. Questions in chat any time; I'll take them at the end, or immediately if it's a quick one."

Also say this, because it relaxes everyone: **"The goal today isn't to make you fluent. It's to make sure you understand what your team is talking about, and to give you five commands you can actually use tomorrow."**

Slide 3 • The problem (3 min)

This is your best non-dev slide. Everyone has lived `report-final-FINAL-v2-USE-THIS.docx`.

Ask the room a question here — it's the right moment for engagement, early:

"Quick one in chat — who's got a file on their machine right now with the word 'final' in the name twice?"

Then land the three failures of the zip-folder approach: you don't know which is current, you can't merge two people's edits, and there's no record of who changed what or why. Git fixes exactly those three.

Slide 4 • What is Git (3 min)

Keep the history short — Linus Torvalds, 2005, built it in a couple of weeks because the Linux kernel team lost access to the tool they'd been using. It's a nice story but don't spend more than 30 seconds.

Spend your time on the two words that matter: **distributed** (Part 1.4 above) and **version control** (it records history; it doesn't compile, test, or deploy anything).

Worth saying plainly: "Git doesn't know or care what's in your files. It works on text files, images, spreadsheets, anything. It's just much better at text."

Slide 5 • Git vs GitHub vs GitLab (3 min)

The highest-value slide for non-devs. They hear "GitHub" in every startup and think it's the same thing as Git.

"Git is the engine. GitHub is a car built around that engine, with seats and a dashboard. GitLab is a different car with the same engine. There's also Bitbucket, and Azure DevOps. All of them run Git underneath — that's why the commands are identical no matter where your company hosts code."

The practical version: *Git works with no internet and no account. GitHub is where you put it so other people can see it, review it, and discuss it.*

Slide 6 • The mental model (4 min — go slow)

Use Part 1.7. Do not rush this slide even if you're behind. Say each transition out loud and point at it:

1. "I edit a file. Git notices, but has done nothing."
2. "`git add` — I'm saying *this change belongs in my next save point*."
3. "`git commit` — I'm sealing that save point into my local history. Still nobody else can see it."
4. "`git push` — now the team has it."

Then the reverse: "`git pull` brings their work down to me."

The sentence to repeat twice: **"Commit is local. Push is what makes it real for everyone else."** That confusion causes more "but I committed it!" incidents than anything else on the slide.

Slide 7 • The everyday loop (4 min)

Frame it as: "If you remember nothing else, remember this cycle. I have used Git professionally for years and this is 90% of what I type."

Emphasise `git status` disproportionately. It is the safest command in Git — it changes nothing, and it tells you exactly where you are and usually what to do next. Tell the room: *"When you're confused, run `git status`. It's free and it's often the answer."*

Slide 8 • Setup and first repo (6 min)

You have generous time here — use some of it for the first live demo (Part 3). If you'd rather keep demos together, just walk the commands and note:

- The `user.name` and `user.email` you set here get baked into every commit you ever make. Commits are signed with that identity permanently — worth getting right.
- `--global` means "for every repository on this machine." Drop it inside a repo to override just there (useful if you use a work email at work

and a personal one for side projects).

- `git init` vs `git clone`: `init` creates a brand-new empty repository in a folder you already have. `Clone` copies an existing one, with all of its history, from a server. **You will use clone about fifty times for every one time you use init.**

Slide 9 • The twelve commands (4 min)

Don't read all twelve. Read the first five slowly (`status`, `add`, `commit`, `push`, `pull`) and then say: "The rest are on the slide, and I'll come back to branching and merging properly in a moment. Screenshot this."

One useful aside: `git switch` and `git checkout` do overlapping things. `checkout` is the old command that did about six unrelated jobs, which is why it confused everyone. Git split it into `switch` (change branches) and `restore` (undo file changes). Both still work — if the room sees `checkout` in older tutorials, it's not wrong, just older.

Slide 10 • Commit messages (3 min)

Good engagement moment — invite worst-commit-message confessions in chat.

Give the practical rule rather than a philosophy: **"Write the message so that it completes the sentence 'If applied, this commit will...'"** That's why imperative present tense — "Add login validation," not "Added login validation." It reads correctly in the log, in release notes, and in the PR list.

And explain *atomic* in business terms, because non-devs will care: "If a commit does one thing, we can undo exactly that one thing when it breaks in production at 2am. If a commit does five things, we have to undo all five."

Slide 11 • Branching (4 min)

Use the bookmark analogy from Part 1.3, then the practical framing:

"A branch is a safe place to be wrong. You can break anything you like on a branch — `main` is untouched, and nobody sees your mess until you choose to show them."

Mention the naming convention because it's a team decision they'll actually be asked about: `feature/`, `bugfix/`, `hotfix/`, often with a ticket number — `feature/JIRA-1234-login-validation`. The reason is searchability and automation, not aesthetics; CI systems and tools key off those prefixes.

Slide 12 • Merging (3 min)

The one distinction to make clear:

- **Fast-forward:** `main` hasn't moved since you branched. Git just slides the `main` bookmark forward to your commit. No merge commit, no conflict, nothing to resolve.
- **Three-way merge:** both branches moved. Git looks at three points — your branch tip, their branch tip, and the common ancestor where you split — and works out a combined result. It succeeds automatically most of the time.

Say clearly: **"Git merges by line, not by meaning."** Two people editing different lines of the same file merges cleanly with no help. Two people editing the *same line* is a conflict. That's the whole rule, and it sets up the next slide.

Slide 13 • Merge conflicts (4 min)

The most feared topic and the easiest to defuse. Lead with reassurance:

"A conflict is not an error. Nothing is broken and nothing is lost. Git is saying: two people changed the same line, and I'm not willing to guess which one you meant. You decide."

Walk the markers literally:

- `<<<<<<< HEAD` — everything below this is **your** version (the branch you're on)
- `=====` — the divider
- `>>>>>>> feature/dashboard` — everything above this is **their** version (the branch coming in)

Then the fix: open the file, decide what the final text should be (it might be yours, theirs, or a combination you type fresh), **delete all three marker lines**, save, test, `git add` the file, `git commit`.

Two things worth adding:

- `git merge --abort` puts everything back exactly as it was before you started the merge. It's a free escape hatch, and most people don't know it exists.
- Modern editors (VS Code especially) render conflicts with "Accept Current / Accept Incoming / Accept Both" buttons, so you often never see raw markers. Mention this — it visibly relaxes the room.

Slide 14 • Pull requests (3 min)

Reframe it for the non-devs, who *will* interact with PRs even if they never write code:

"A pull request is a formal request: 'I've done some work on my branch — please review it and, if you're happy, put it into the main project.' It's a discussion thread attached to a set of changes. Reviewers comment, the automated tests run, and when everyone's satisfied, someone clicks merge."

Point out the naming: GitHub says *Pull Request*, GitLab says *Merge Request*. Identical thing. GitLab's name is arguably more accurate.

Mention the three merge buttons GitHub offers, since it's a real team decision:

- **Merge commit** — keeps every commit plus a merge commit. Full history.
- **Squash and merge** — flattens your branch's commits into one tidy commit on `main`. Most popular choice for teams.
- **Rebase and merge** — replays commits onto main with no merge commit.

Slide 15 · Fetch vs pull (3 min)

Use the tracking-branch explanation from Part 1.5, then the email analogy on the slide. Repeat the analogy twice — it's on your own speaker notes and it's good advice.

The practical takeaway: **"Use `pull` by default. Use `fetch` when you want to look before you leap — for example when you know a big change landed upstream and you want to read it before it touches your files."**

Slide 16 · Undoing mistakes (4 min)

Slow down here; this is the slide that prevents incidents.

The organising question: **"Has anyone else seen this commit?"**

- **No, it's only on my machine** → `reset` is available to you. `git reset --soft`, `--mixed`, `--hard` move your branch pointer backwards; `--hard` also throws away your file changes.
- **Yes, it's been pushed** → use `git revert`. Revert doesn't delete anything. It creates a *new* commit that is the exact opposite of the old one. History stays intact, everyone's copies stay compatible, and the bad change is neutralised.

Then the safety net nobody knows about, which is a great thing to give a room:

" `git reflog` . Git keeps a private log of everywhere HEAD has been for the last 90 days, including commits you 'deleted'. If you `reset --hard` and panic, `git reflog` usually gets it back. It has saved me and it will save you."

Slide 17 · Log and diff (2 min)

Quick slide. The one habit to sell: `git diff --staged` **before every commit**. It's a five-second self-review that catches debug statements, commented-out code, and accidentally staged files. Frame it as "reading your own work before you submit it."

Useful extras if asked: `git log --oneline --graph --all` draws the branch structure as ASCII art; `git log -- <filename>` shows the history of one file; `git blame <file>` shows who last changed each line (and why — it gives you the commit).

Slide 18 · Case study (5 min)

This is where it all clicks. Narrate it as a story, in first person, with a real-sounding ticket:

"It's Monday. I pick up JIRA-1234, add login validation. First thing — `git switch main`, then `git pull`, because I want to start from what the team has, not from Thursday's stale copy. Then I branch: `git switch -c feature/JIRA-1234-login-validation`. I write the code, I test it. `git status` to see what I touched, `git diff` to read it. `git add`, `git commit` with a message that says what it does. `git push -u origin feature/...` I open a PR, tag two reviewers, link the ticket. They comment, I push two more commits addressing their comments, CI goes green, they approve, we squash-merge, I delete the branch. Tomorrow morning I `git pull` on main and my own change comes back down to me as part of the shared history."

That paragraph is the single most useful 90 seconds of the session for developers.

Slides 19-20 · Best practices and anti-patterns (7 min combined)

Don't read 20 bullets aloud — you'll lose the room. Pick five you have an opinion about and tell a short story for each. Suggested five:

1. **Never commit secrets**. The critical detail: deleting the file in a later commit does *not* remove it from history. Anyone who clones the repo gets the whole history, including the commit with your key in it. The only real fix is to rotate the key — assume it's compromised the moment it's pushed. This is the one to say twice.
2. **Pull before you start**. Ninety percent of painful conflicts come from branching off a week-old copy of main.
3. **Small commits**. Ask the room to imagine reviewing a 600-line change vs six 100-line changes. Reviewers skim large PRs; that's where bugs get through.
4. **Don't force-push shared branches**. `--force` rewrites history on the server. Anyone who already pulled now has a history that doesn't match, and their next push either fails or creates a mess. If you truly need it, `--force-with-lease` is the safer version — it refuses if someone else has pushed since you last looked.
5. **Protect `main`**. On GitHub this is a setting: require a PR, require an approval, require CI to pass. It turns "please don't commit to main" from a team norm into something the tool enforces. Worth noting this is your decision as tech lead, and you can enable it this week.

Also introduce `.gitignore` here if you haven't — it's the file that tells Git "never track these." Typically `node_modules/`, `.env`, `dist/`, `build/`, `.DS_Store`, IDE folders. Non-devs get this instantly: "a list of things not to file."

Slides 21-22 · Demo and cheat sheet

Demo content is Part 3 and Part 4 below. Then tell people explicitly to screenshot slide 22.

Slides 23-24 · Takeaways and Q&A (10-15 min)

Read the eight takeaways aloud — it's genuinely the right call for retention. Then open the floor with a specific prompt, because "any questions?" in a virtual room gets silence:

"Let me start with the one I get most often, and then I'll take yours..." — and answer a question from Part 6 yourself. That breaks the ice reliably.

PART 3 — Live demo A: the terminal walkthrough (6-8 min)

Rehearse this end-to-end at least once before the session. A demo that fails live costs you more credibility than not demoing at all.

Setup before the call

- Terminal font size at 18–20pt minimum. Nobody can read your normal size on a shared screen.
- Use a light theme or a very high-contrast dark one — compression on video calls destroys low-contrast text.
- Start in an empty folder. Clear your terminal history.
- Close Slack, email, and anything with notifications.
- Have a browser tab open and signed in to GitHub.

The script

```
mkdir git-demo && cd git-demo
git init
git status
```

Narrate: "Notice it says 'No commits yet' and 'nothing to commit'. Git is watching this folder now, but there's no history."

```
echo "# Login form" > login.js
git status
```

Narrate: "Now `login.js` shows up in red under 'Untracked files'. Git can see it, but it isn't managing it yet. Note the hint Git gives you — it's literally telling you to run `git add`."

```
git add login.js
git status
```

Narrate: "Same file, now green, now under 'Changes to be committed'. That's the staging area from our diagram. Nothing has been saved to history yet — I can still back out."

```
git commit -m "Add initial login form"
git log --oneline
```

Narrate: "There's our first commit, with its short SHA. That's a permanent, recoverable point in history."

Now branching:

```
git switch -c feature/validation
echo "validate(user)" >> login.js
git add .
git commit -m "Add user validation to login form"
git log --oneline
```

Narrate: "Two commits on this branch. Watch what happens when I go back to main."

```
git switch main
cat login.js
```

Narrate — this is the moment of the demo: "Look at the file. My validation line is *gone*. It's not lost — it's on the other branch. Git rewrote the files on disk to match this branch. This is what people mean when they say branches are parallel universes."

```
git merge feature/validation
cat login.js
git log --oneline --graph
```

Narrate: "The line is back. And notice Git said 'Fast-forward' — main hadn't moved, so it just slid the pointer forward. No merge commit needed."

Optional: create a conflict on purpose (2 min, only if you're ahead of schedule)

```
git switch -c branch-a
echo "Welcome to Dashboard" > home.txt
git add . && git commit -m "Add dashboard welcome"

git switch main
echo "Welcome to User Dashboard" > home.txt
git add . && git commit -m "Add user dashboard welcome"

git merge branch-a
```

Git will refuse and report a conflict. Open `home.txt` in your editor on screen, show the markers, delete them, keep one line, save, then:

```
git add home.txt
git commit -m "Resolve welcome message conflict"
```

Narrate: "That's it. That's a merge conflict, start to finish, in under a minute. It's not a disaster — it's Git asking a question and me answering it."

PART 4 — Live demo B: deploy a live site with GitHub Pages (8-10 min)

This is your best material and the thing people will remember. In under ten minutes the room watches a text file on your laptop become a public URL. It makes Git feel consequential rather than administrative.

Rehearse it. Then delete the repo and do it fresh on the call. And have a fallback ready (see below).

Step 1 — Create the repository on GitHub (1 min)

On github.com: **New repository** → name it `git-demo-site` → **Public** (Pages needs public on free accounts) → tick **Add a README** → Create.

Narrate what you're doing: "This is the remote repository from our diagram — the shared copy that lives on a server."

Step 2 — Clone it (1 min)

Copy the HTTPS URL from the green **Code** button.

```
git clone https://github.com/<your-username>/git-demo-site.git
cd git-demo-site
ls
git log --oneline
```

Narrate: "I now have the full history on my machine — in this case, one commit that GitHub made when it created the README."

Step 3 — Build the site (2 min)

```
cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Git Training Demo</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 640px;
      margin: 80px auto; padding: 0 20px; line-height: 1.6; }
    h1 { font-size: 2.5rem; margin-bottom: 0; }
    .tag { color: #666; }
  </style>
</head>
<body>
  <h1>It's live.</h1>
  <p class="tag">Deployed from a Git commit during the DTSS Git workshop.</p>
  <p>Version 1</p>
</body>
</html>
EOF

git status
git add index.html
git commit -m "Add landing page"
git push
```

Narrate each step against the four-box diagram. "Edit — working directory. Add — staging. Commit — local repository. Push — remote. That's the same four boxes from slide 6, and now you're watching them happen."

Step 4 — Turn on Pages (1 min)

In the repo on GitHub: **Settings** → **Pages** (left sidebar) → under *Build and deployment*, Source = **Deploy from a branch**, Branch = **main**, folder = **/ (root)** → **Save**.

While it builds (30-90 seconds), fill the gap: go to the **Actions** tab and show the deployment running. "That's CI/CD — automation that triggers on a Git event. You don't need to understand it today, but notice the trigger: *a commit landed on main*. That's the pattern behind every deployment pipeline in the industry."

Step 5 — Show the live URL (1 min)

```
https://<your-username>.github.io/git-demo-site/
```

Open it on screen. Pause. Let it land.

Step 6 — Ship a change live (2 min)

```
sed -i 's/Version 1/Version 2 – updated live during the session/' index.html
git add .
git commit -m "Update version text"
git push
```

Wait ~60 seconds, hard-refresh the browser (Ctrl+Shift+R), and it's changed.

The closing line: "That's the entire idea. A commit isn't paperwork — a commit is how software gets to users. Everything we covered today is the machinery that makes that safe when forty people are doing it at once."

Fallback plan

Pages occasionally takes several minutes on first deploy. Have a second repo, already deployed and working, bookmarked. If the live one stalls, say "this sometimes takes a couple of minutes to propagate — here's one I deployed yesterday, same six steps," and show that. Nobody minds. Also note: GitHub's own web editor can do the whole thing without a terminal, which is a useful thing to show non-devs anyway (see Part 5.7).

PART 5 — Bonus material if you have time to fill

Your agenda is tight, but virtual sessions sometimes run fast because there's less interruption. Keep these in your back pocket, each is 2–4 minutes.

5.1 `git stash` — the "I need to switch right now" command

You're halfway through a change and an urgent bug comes in. You don't want to commit half-finished work.

```
git stash          # shelve all uncommitted changes, clean working tree
git switch main    # go fix the urgent thing
# ... fix, commit, push ...
git switch feature/x
git stash pop      # bring your half-done work back
```

`git stash list` shows what you've shelved. Framing: "a drawer you sweep your desk into."

5.2 Tags and releases

A tag is a permanent name for a specific commit — usually a version number.

```
git tag -a v1.2.0 -m "Release 1.2.0"
git push origin v1.2.0
```

Branches move; tags don't. That's the entire distinction. On GitHub, tags become Releases, which is how you ship downloadable versions. Good for non-devs: "it's the difference between a bookmark that moves as you read and a permanent label on a specific page."

5.3 `.gitignore`, properly

```
node_modules/
.env
*.log
dist/
.DS_Store
.vscode/
```

The gotcha worth mentioning: `.gitignore` only affects **untracked** files. If you already committed `.env`, adding it to `.gitignore` changes nothing — Git keeps tracking it. You need `git rm --cached .env` and a commit. And even then, it's still in history. Rotate the secret.

5.4 `git blame` — not as hostile as it sounds

```
git blame index.html
```

Shows, line by line, which commit last touched it and who wrote it. Its real use isn't blame, it's archaeology: "why is this weird line here?" → find the commit → read the message → read the PR → understand the decision. This is the strongest argument for good commit messages, so it pairs well with slide 10.

5.5 Fork vs clone

- **Clone** — copy a repo to your machine. You need write access to push back.
- **Fork** — make your own copy of someone else's repo *on GitHub*, under your account. You then clone your fork, push to your fork, and open a PR back to the original.

Forks are how open source works with strangers. Inside a company you usually just use branches, because everyone has write access.

5.6 Cherry-pick

```
git cherry-pick <sha>
```

Takes one specific commit from anywhere and applies it to your current branch. Real use: a bug fix landed on `develop` and you need exactly that fix on the release branch without dragging along everything else.

5.7 GitHub without a terminal — a non-dev track (worth 4 minutes)

Genuinely valuable for half your audience. Show these in the browser:

- **Edit a file directly on GitHub** — open any file, click the pencil icon, edit, scroll down, write a commit message, click Commit. You just made a commit without touching a terminal. GitHub can even create a branch and open a PR for you in the same dialog.
- **Issues** — the ticket system. Anyone can file one; developers link commits and PRs to them.
- **Projects** — kanban boards that stay in sync with issues and PRs.

- **The README** — the front page of a repo. Written in Markdown. Non-devs edit these constantly.
- **Reading a PR** — click the *Files changed* tab. Green is added, red is removed. You can comment on any specific line. Show this: it's how a non-technical stakeholder can follow what's being shipped.

Say it explicitly: **"If you're not a developer, this is your Git. You never need the terminal."**

5.8 What a good team workflow looks like (tech-lead framing)

Since you're the tech lead, a slide-less two minutes on your actual team convention is high-value and nobody else can deliver it:

- `main` is always deployable and is protected.
- All work happens on branches named `feature/TICKET-description`.
- Every change goes through a PR with at least one approval.
- CI must be green before merge.
- Squash-merge into main, then delete the branch.
- Tag releases.

Then invite discussion on it. That turns a training session into a team alignment session, which is worth more.

PART 6 — Q&A bank

Prepared answers for what actually gets asked. Skim these the morning of the session.

Conceptual

Q: What's the difference between Git and GitHub?

Git is the version-control software running on your computer. GitHub is a website that stores Git repositories and adds collaboration — pull requests, reviews, issues, automation. You can use Git with no GitHub account at all.

Q: Is Git only for code?

No. Git tracks any file. It's excellent for anything text-based — code, configuration, documentation, Markdown, CSV. It's poor with large binary files like videos or big images, because it can't compress differences between versions and the repo balloons. For those, teams use Git LFS.

Q: Do I lose my work if I switch branches?

No — but Git will refuse to switch if you have uncommitted changes that would be overwritten. Either commit them or `git stash` them first.

Q: What actually is `origin`?

Just a nickname for the remote URL you cloned from. Nothing special about the word. You can rename it or have several remotes.

Q: What's `HEAD`?

A pointer to where you currently are — normally to the branch you've got checked out.

Practical

Q: I committed to the wrong branch. What do I do?

If you haven't pushed: note the commit SHA, `git switch correct-branch`, `git cherry-pick <sha>`, then go back to the wrong branch and `git reset --hard HEAD~1` to remove it there. If you have pushed, do the cherry-pick and use `git revert` on the wrong branch instead of reset.

Q: I committed a password/API key. How bad is it?

Assume it's compromised the moment it's pushed and **rotate the key immediately** — that's the actual fix. Removing it from history is possible (`git filter-repo`, or BFG Repo-Cleaner) but it rewrites every subsequent commit, requires everyone to re-clone, and doesn't help if anyone already pulled it. Rotate first, clean up second.

Q: How do I undo the last commit?

Depends what you want. `git reset --soft HEAD~1` removes the commit but keeps your changes staged — good for "wrong message" or "forgot a file." `git reset --hard HEAD~1` removes the commit and the changes. `git revert HEAD` creates a new opposite commit — the only safe option if you've pushed. Also: `git commit --amend` fixes the most recent commit's message or contents, as long as it isn't pushed yet.

Q: My push was rejected. Why?

Almost always because someone pushed to that branch after you last pulled. Run `git pull`, resolve anything that conflicts, then push again. Resist the urge to use `--force`.

Q: `git pull` gave me a merge conflict and I panicked. Can I go back?

Yes. `git merge --abort` restores the state from before the pull.

Q: What's the difference between `git reset` and `git revert`?

Reset moves your branch pointer backwards — history changes, commits can disappear. Revert adds a new commit that undoes an old one — history is preserved. On anything shared, use revert.

Q: I deleted a branch that had work on it. Is it gone?

Probably not. `git reflog` will show where it pointed; `git switch -c recovered-branch <sha>` brings it back. Git keeps unreferenced commits for about 90 days.

Team / process

Q: Merge or rebase?

See Part 1.6. Short answer for a team: merge into `main`, use squash-merge on the platform for a clean history, and only rebase branches nobody else has pulled.

Q: How long should a branch live?

Days, not weeks. The longer a branch lives, the further `main` drifts, and the worse the eventual merge. If a piece of work takes three weeks, find a way to merge safe partial pieces along the way.

Q: Should we commit directly to main?

No. Use a branch and a PR even for one-line changes — it gives you review, CI, and an audit trail. Enforce it with branch protection rather than relying on discipline.

Q: How often should I commit?

Whenever a logical unit of work is done and in a working state. Several times a day is normal. Push at least once a day — an uncommitted or unpushed day of work is a day of work living only on your laptop.

Q: Do I need to memorise all these commands?

No. Five commands cover most days: `status`, `add`, `commit`, `push`, `pull`. Everything else you look up when you need it — and so does everyone else, including me.

Awkward ones

Q: Isn't Git overkill for our small team?

Two people is enough to need it. Even solo it's worth it — the history, the ability to try something risky on a branch, and the off-machine backup all pay for themselves quickly.

Q: Why is Git so confusing?

Honest answer, and a good one to give: because the command names were designed around Git's internals rather than around what users are trying to do. `checkout` used to do six unrelated jobs. The newer commands `switch` and `restore` exist specifically to fix that. It's not you.

Q: Something I don't know.

Say: *"Good question — I don't want to guess at that one. Let me check and come back to you in the channel after the session."* Then actually do it. In a mixed room this builds more credibility than a confident wrong answer, and it also models the behaviour you want from your team.

PART 7 — Pre-flight checklist

Day before

- [] Read Part 1 properly
- [] Run Demo A end-to-end in a scratch folder
- [] Run Demo B end-to-end; confirm the Pages URL loads
- [] Create and deploy the *fallback* Pages site; bookmark it
- [] Delete the demo repo so you can create it fresh on the call

Thirty minutes before

- [] Terminal font 18pt+, high contrast, cleared history
- [] Browser signed in to GitHub, zoom at ~125%
- [] Notifications off (Slack, email, OS-level Do Not Disturb)
- [] Deck open, this handbook on a second screen
- [] Empty `~/demos` folder ready to `cd` into
- [] Test screen share and audio

During

- [] State up front that questions go in chat
- [] Ask someone to watch chat for you if the group is large
- [] Record the session if your team allows it — people will rewatch the demos
- [] Watch the clock at slide 14; that's your halfway marker

After

- [] Share the deck, this handbook, and the demo repo URL
 - [] Answer anything you parked
 - [] One short follow-up: "pick one practice from slide 19 to adopt this sprint"
-

PART 8 — Glossary to paste in chat

Term	Meaning
Repository (repo)	A project folder that Git is tracking, plus its full history
Commit	A saved snapshot of the project at a moment in time
SHA / hash	The unique fingerprint ID of a commit
Branch	A movable pointer to a commit — a parallel line of work
HEAD	Where you currently are
main	The primary branch, by convention the stable one
Remote	A hosted copy of the repo, usually nicknamed <code>origin</code>
Clone	Download a full copy of a repo, history included
Fork	Your own copy of someone else's repo on GitHub
Staging area	The basket of changes going into your next commit
Merge	Combine one branch's work into another
Conflict	Two branches changed the same lines; a human must choose
Pull request (PR)	A request to merge your branch, with review and discussion
Fetch	Download remote changes without applying them
Pull	Fetch, then merge into your current branch
Push	Upload your commits to the remote
Tag	A permanent label on a commit, usually a version number
.gitignore	A list of files Git should never track
Stash	Temporarily shelve uncommitted changes
CI/CD	Automation that runs on Git events — tests, builds, deploys

PART 9 — Links to share afterwards

- **Pro Git book** (free, official, excellent) — <https://git-scm.com/book/en/v2>
 - **Learn Git Branching** — <https://learngitbranching.js.org> — interactive visual sandbox, the single best link for anyone who wants to actually practise
 - **Oh Shit, Git!?!** — <https://ohshitgit.com> — a short list of "I broke it, how do I fix it" recipes
 - **GitHub Skills** — <https://skills.github.com> — hands-on guided courses inside GitHub itself
 - **GitHub Pages docs** — <https://docs.github.com/en/pages>
 - **Visualizing Git** — <https://git-school.github.io/visualizing-git/> — type commands, watch the graph move
-

One last thing

You said you don't know Git deeply. Neither do most people presenting it — and your deck is better than most professional training material I've seen. The concepts in Part 1 are genuinely all you need on top of it. If you can explain the four boxes on slide 6, the fact that a branch is just a pointer, and the difference between `reset` and `revert`, you will be the most Git-literate person in that meeting.